A. Actual Source Code

Snippet 1 - Scholarship Eligibility Filtering

Source file: backend/scholarship_rules.py | Function: recommend_scholarships

Original line numbers: 146-157 | 12 original lines

```
146 |     for item in scholarships:
147 |         if item.get("applicationsBlocked") is True:
148 |             continue
149 |         if item.get("applicationEnabled") is False and item.get("applyOpen") is not True:
150 |             continue
151 |
152 |         minimum_gwa = _to_float(item.get("minimumGwa") or item.get("minGwa") or item.get("minimumGrade"))
153 |         provider = item.get("providerLabel") or item.get("grantorName") or item.get("title") or item.get("name") or ""
154 |         if minimum_gwa is None:
155 |             minimum_gwa = _to_float(get_scholarship_policy(provider)["minGwa"]) or 2.25
156 |         if student_gwa is None or student_gwa > minimum_gwa:
157 |             continue
```

This is one continuous extract. Input preparation on lines 137-145 and the following scoring and result construction are outside this extract.

Suggested caption: Figure 3.X. Recommendation-stage filtering using application availability and minimum-GWA conditions.

Snippet 2 - Weighted Recommendation Scoring

Source file: backend/scholarship_rules.py | Function: recommend_scholarships

Original line numbers: 160-161; 167-169; 173; 181-185 | 11 original lines

```
160 |         grade_margin = max(0.0, minimum_gwa - student_gwa)
161 |         grade_score = min(40.0, 24.0 + (grade_margin * 12.0))
```

[Documentation annotation: original lines 162-166 omitted; these contain reason-label handling.]

```
167 |         roster_count = _to_int(item.get("rosterCount"))
168 |         popularity_weight = 30.0 if is_high_grade_student else 22.0
169 |         popularity_score = (roster_count / max_roster * popularity_weight) if max_roster > 0 else 0.0
```

[Documentation annotation: original lines 170-172 omitted; these contain reason-label handling.]

```
173 |         location_score = _location_score(student, item)
```

[Documentation annotation: original lines 174-180 omitted; these contain reason-label handling.]

```
181 |         completeness_score = 8.0
182 |         if item.get("profileImageUrl") or item.get("authorImageUrl") or item.get("imageUrl"):
183 |             completeness_score += 2.0
184 |
185 |         score = grade_score + popularity_score + location_score + completeness_score
```

Four separate original ranges are shown. Lines 162-166, 170-172 and 174-180 are omitted and explicitly marked; no replacement code or pseudocode is inserted.

Suggested caption: Figure 3.Y. Weighted recommendation-score calculation using GWA fit, roster popularity, location fit and image availability.

Snippet 3 - Scholarship Ranking

Source file: backend/scholarship_rules.py | Function: recommend_scholarships

Original line numbers: 187-204 | 18 original lines

```
187 |         recommendations.append(
188 |             {
189 |                 "item": item,
190 |                 "score": round(score, 4),
191 |                 "label": label,
192 |                 "reasons": reasons[:4],
193 |                 "criteria": {
194 |                     "gwa": student_gwa,
195 |                     "minimumGwa": minimum_gwa,
196 |                     "rosterCount": roster_count,
197 |                     "gradeScore": round(grade_score, 2),
198 |                     "popularityScore": round(popularity_score, 2),
199 |                     "locationScore": round(location_score, 2),
200 |                 },
201 |             }
202 |         )
203 |
204 |     recommendations.sort(key=lambda row: row["score"], reverse=True)
```

This is one continuous extract. Earlier score calculation and the later API response wrapper are outside this extract.

Suggested caption: Figure 3.Z. Construction of recommendation results and descending ranking by the computed score.

B. Short Technical Explanation

Snippet 1 - Scholarship Eligibility Filtering

This loop skips scholarships when applications are blocked, when the enabled/open flags fail the shown condition, or when the student's GWA is missing or exceeds the required minimum. A missing scholarship GWA requirement is resolved using the provider policy; these are recommendation-stage filters, not the complete application-approval checks.

Snippet 2 - Weighted Recommendation Scoring

The code adds GWA fit, roster popularity, location fit and a fixed completeness component with an image bonus to obtain one ranking score. The location calculation is delegated to _location_score; the high-grade flag and maximum roster denominator are prepared earlier on lines 141-143 and are explained in Section C.

Snippet 3 - Scholarship Ranking

Each eligible scholarship is stored with its score rounded to four decimal places and with explanatory metadata. The final sort places higher stored scores first; equal stored scores retain their incoming order because Python's list sort is stable.

Verified call path and boundaries

The student recommendation loader calls recommendScholarshipsWorkflow, which posts to /scholarships/recommend. The endpoint authenticates the student, loads that student's stored record, and invokes recommend_scholarships. This extraction documents the weighted backend ranker; it does not substitute the separate duplicate-detection algorithm or the administrator's student-matching calculation.

Call references: src/services/recommendedScholarshipService.js:195; src/services/recommendedScholarshipService.js:280; src/services/workflowService.js:100; backend/main.py:797

Scope of evidence: read-only source inspection and exact line extraction from this checkout. No claim is made about deployed code, measured accuracy, real student outcomes or prediction performance. No credentials, runtime records or student data were read for this task.

C. Algorithm Verification

Mandatory recommendation-stage conditions

Condition	Exact behavior	Source lines
Applications blocked	Exclude when applicationsBlocked is exactly True.	147-148
Enabled/open condition	Exclude when applicationEnabled is exactly False AND applyOpen is not exactly True. applyOpen=True overrides that particular disabled test; applyOpen=False alone is not tested here.	149-150
Student GWA	The parsed student GWA must not be None and must be <= minimum_gwa. Lower numeric GWA is treated as better. This function does not check uploaded documents.	141, 156-157
Scholarship GWA	Read minimumGwa, minGwa or minimumGrade. If conversion fails or no value is available, use the provider policy; its final default is 2.25.	152-155

Configured fallback GWA policies

get_scholarship_policy resolves provider names to these static defaults: Tina Pancho 2.00, Morisson/Morrison 2.25, Kuya Win 1.75, and Other 2.25. An explicit scholarship GWA requirement takes precedence; the frontend candidate normalizer can already supply a 2.25 default before the backend receives the candidate. References: backend/scholarship_rules.py:6-11; backend/scholarship_rules.py:15-28; src/services/recommendedScholarshipService.js:147-159, 247-255.

What these filters do not prove

The recommendation function itself does not enforce COR/ROG/profile approval, account activation, income, course, year level, available slots, age, scholarship commitment or the grantor's permitted province-municipality pairs. Do not describe these as weighted inputs or mandatory checks inside this function. Application submission has a separate reservation workflow. backend/scholarship_rules.py:137-216; backend/workflow_service.py:476; backend/scholarship_choice_service.py:72.

Availability nuance: the backend uses the exact boolean conditions in Snippet 1; it does not simply require applyOpen=True for every raw caller-supplied record. The normal student loader applies additional availability filtering before the ranking call. Do not turn the response's short hardFilters labels into stronger checks than the actual conditions shown.

C. Actual Weights and Formulas

Factor	Calculation	Weight / interpretation	Lines
GWA fit	margin = max(0, minimum_gwa - student_gwa); grade_score = min(40, 24 + margin * 12)	24 to 40 for eligible ordinary inputs	160-161
Roster popularity	popularity_score = roster_count / max_roster * popularity_weight when max_roster > 0; otherwise 0	22-point weight; 30 if student GWA <= 1.75	142-143, 167-169
Location fit	location_score = _location_score(student, item)	Up to 25; exact rules below	173, 239-262
Completeness/image	completeness_score = 8; add 2 when profileImageUrl, authorImageUrl or imageUrl is truthy	8 base; 10 with an image	181-183
Total	score = grade_score + popularity_score + location_score + completeness_score	Additive raw ranking score, not a percentage	185

Location helper: _location_score

Rule	Implemented calculation	Lines
No location data anywhere	Return 8 only if all six normalized student/grantor province, city and barangay values are empty.	240-248
Province component	Same nonempty province: +10. Otherwise, student Pampanga and grantor Bulacan: +4. Otherwise, any nonempty grantor province: +2. Only one province branch applies.	250-256
City component	Same nonempty city: +10. This comparison is independent of the province branch.	258-259
Barangay component	Same nonempty barangay: +5. This is a ranking bonus, not a barangay-based location-scope permission check.	260-261
Cap/normalization	Return min(score, 25). Normalization lowercases and replaces non-ASCII-letter/digit runs with spaces, then trims; it is not a distance calculation or a Baliwag/Baliuag alias resolver.	234-236, 262

C. Ranking and Interpretation

Ranking method

The function stores round(score, 4) and performs a descending list sort on the stored score. There is no explicit secondary key, so equal rounded scores retain input order; changing the input order can therefore change the order of tied recommendations. References: backend/scholarship_rules.py:190, 204.

Algorithm type

The implementation is a deterministic, predefined-rule weighted scoring and ranking mechanism. This recommendation path does not train or load a machine-learning model and does not construct an explicit decision-tree model. The source identifies it as Weighted Recommendation Scoring, not a learned Decision Tree. References: backend/scholarship_rules.py:137-216.

The score is not an acceptance probability

The raw factor budgets total up to 97 points with the normal popularity weight, or 105 points with the high-grade popularity weight, for ordinary nonnegative roster inputs. No conversion to a probability or normalization to 100 is performed, so the score must not be described as an acceptance percentage, accuracy rate or official scholarship selection rank. A score of at least 78 selects the label Best Scholarship Match For You; that is a display-label threshold, not an approval threshold. References: backend/scholarship_rules.py:160-185, 190; backend/scholarship_rules.py:265-274.

Interpretation cautions

The roster-popularity denominator is computed across all supplied candidates before per-candidate filtering. Even a candidate later excluded can influence that denominator. The loader's rosterCount counts non-archived scholar rows per grantor; it is not a measured acceptance rate, number of clicks or number of applications. backend/scholarship_rules.py:141-143; src/services/recommendedScholarshipService.js:207-213.

The field called profileCompleteness is actually a constant eight-point component plus a two-point image-presence bonus. It is not a student-profile completeness percentage and does not score income, course, year, achievements or document completeness. backend/scholarship_rules.py:181-183, 213.

C. End-to-End Verification Notes

Candidate preparation

Candidate preparation is separate from numeric scoring. The loader excludes archived/inactive/disabled grantor accounts and uses actual open, non-hidden, non-archived scholarship announcements. It requires a grantor ID, applicationEnabled and applicationsBlocked != true before sending candidates to the backend. src/services/recommendedScholarshipService.js:195; src/services/recommendedScholarshipService.js:147-176, 223-276.

Authenticated student data

The authenticated recommendation endpoint replaces the submitted student object with the signed-in student's stored row, then calls the scoring function. The candidate list still comes from the caller; a ranking response is not final application authorization. backend/main.py:797; src/services/workflowService.js:100.

Local fallback

On backend_unavailable or request_timeout, the browser uses a local weighted-scoring fallback and marks the response degraded. It does not fall back for every API error. The local fallback defaults a missing minimum GWA to 2.25, whereas the backend can select a provider-specific policy; do not claim the paths are identical for every possible input. src/services/recommendedScholarshipService.js:76; src/services/recommendedScholarshipService.js:285.

Student-page ordering

The student display is not necessarily a pure score-sorted list. It deduplicates by scholarship and merges invitations/reapplication items (priority 1), admin recommendations (priority 2) and algorithm results (priority 3, or 2 when matched-grantor context exists), then sorts by priority before score. With SCHOLARSHIP_CHOICE_ENABLED=true, getGrantorApplicationBlock can hide committed, same-grantor-active or cooldown-blocked options; multiple-scholarship conflicts hide the list. The older allowed-grantor restriction in this page is conditional on that feature being false. src/pages/StudentRecommendedScholarshipsPage.jsx:150; src/pages/StudentRecommendedScholarshipsPage.jsx:217; src/services/scholarshipChoiceService.js:3; src/services/scholarshipChoiceService.js:70.

Important distinction: location scoring can include barangay matching and does not calculate kilometers. That does not mean the separate grantor location-scope permission check uses barangays. Likewise, a recommendation is an option to consider, not a reserved slot, commitment or administrator approval.

D. Screenshot Recommendation

Use the three supplied PNGs, one per stage. They contain 12, 11 and 18 original source lines respectively. Snippet 2 has three clearly separated omission bands; its full visual contains 14 code/annotation rows, excluding headings. Do not crop off the original-line gutter or omission labels, and do not present the extracts as standalone executable programs.

Screenshot 1: Figure_1_Scholarship_Eligibility_Filtering.png

Figure 3.X. Recommendation-stage filtering using application availability and minimum-GWA conditions.

Screenshot 2: Figure_2_Weighted_Recommendation_Scoring.png

Figure 3.Y. Weighted recommendation-score calculation using GWA fit, roster popularity, location fit and image availability.

Screenshot 3: Figure_3_Scholarship_Ranking.png

Figure 3.Z. Construction of recommendation results and descending ranking by the computed score.

Replace X, Y and Z with the next consecutive figure numbers used in Chapter III. Place the figures in the Scholarship Recommendation / Weighted Recommendation Scoring discussion and use Section B's matching explanations below them. Use a full-width landscape figure page if necessary to keep long original lines readable; do not shrink all three images onto one page. Do not caption the figures as trained Decision Tree, AI prediction accuracy or automatic scholarship approval.

Traceability and unchanged-source check

Each code block was sliced directly from the stated file and checked against its function's AST boundaries. Full input-file hashes and exact block text are saved in Source_Extraction_Manifest.json. Original code indentation is retained; PNG gutters and PDF line-number prefixes are added annotations rather than source characters. No application source, manuscript, existing guide, database, configuration or environment file was changed.

Source snapshot: 2026-10-25T14:23:04+08:00. Source line numbers may change if the project is edited later. Re-extract before the final manuscript submission if the implementation changes.